UCC 122-1
Cloud Architecture

Lab 2: Designing an initial monolithic architecture (Legacy application) for Digi Bank with Jakarta EE

Cloud Computing Professional License

University of the Mountains

By **Engineer Willy Damtchou**

June 2026

Context

In the previous workshop, students analyzed the context of the Digi Bank project, identified the actors, specified the major functionalities, studied the constraints of the banking sector and understood the interest of a modular monolithic architecture for a financial information system in the structuring phase.

The Digi Bank project aims to establish a secure transactional system enabling the management of customers, accounts, financial operations, compliance controls and a unified point of entry, within a framework of reliability, traceability and reasonable scalability.

This second workshop extends this work by moving from analysis to concrete realization, in the form of a practical workshop where students build the application step by step, respecting a complete technical structure and a logical order of development.

Scenario

Digi Bank's technical team has validated the choice of a modular monolith based on Jakarta EE and now wishes to launch a first operational implementation of the system.

The developers must create the framework of the project, configure the development environment, connect the PostgreSQL database, prepare the WildFly server, develop the first business modules, expose REST access points, and then ensure quality through unit tests and automated functional scenarios.

Your role is to produce a first coherent technical version of the platform, directly executable in IntelliJ IDEA or Visual Studio Code.

Workshop objectives

At the end of this workshop, the student should be able to:

- **A1.1:** Set up a complete Java/Jakarta EE development environment.
- **A1.2:** Create and configure a multi-module Maven project.
- **A1.3:** Structure a modular monolithic application around the domains shared, customer, account, transaction, compliance and index.
- **A1.4:** Configure PostgreSQL and WildFly for application execution.
- **A1.5:** Develop REST entities, services, repositories and resources.
- **A1.6:** Run JUnit unit tests.
- **A1.7:** Write and launch BDD scenarios with Cucumber.
- **A1.8:** Deploy the application and verify that it is working correctly.

Resources and prerequisites

a — Reference resources

The choices made in this workshop are based on the framework elements of the Digi Bank project, including the modular structure of the system, the use of Jakarta EE, the organization into distinct functional components, and the use of technologies such as Maven, PostgreSQL, JUnit, Cucumber, and WildFly.[1][3][4]

b — Technical prerequisites

The student must have:

- JDK 17 or a version compatible with Jakarta EE
- Apache Maven
- PostgreSQL
- WildFly
- IntelliJ IDEA or Visual Studio Code
- Git

- A machine with a configuration sufficient to run all of these components locally

1. Environmental preparation

1.1. Installing the tools

Install the following components:

- Java JDK 17
- Maven
- PostgreSQL
- WildFly
- IntelliJ IDEA Community or Ultimate, or Visual Studio Code with Java extensions
- Git

1.2. Basic checks

Execute the following commands in a terminal:

```
Java -version

mvn -version

git --version

psql --version
```

1.3. Expected Results

```
user@ca:4f:12:a0:b3:c1 ~ %  Java -version

mvn -version

git --version

psql --version

Java version "17.0.16" 2025-07-15 LTS

Java(TM ) SE Runtime Environment ( build 17.0.16+12-LTS-247 )

Java HotSpot ( TM ) 64-Bit Server VM (build 17.0.16+12-LTS-247,
mixed fashion, (sharing )

Apache Maven 3.9.11 (3e54c93a704957b63ee3494413a2b544fd3d825b)

Maven home: /opt/homebrew/Cellar/maven/3.9.11/libexec

Java version: 17.0.16, vendor: Oracle Corporation, runtime:
/Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Home

Default local: fr_FR, platform encoding: UTF-8

BONE name: "mac os x", version: "26.5.1", arch: "aarch64",
family: "mac"

git version 2.50.1 (Apple Git-155 )

psql (PostgreSQL) 18.4 ( Homebrew )

user@ca:4f:12:a0:b3:c1 ~ %
```

2. Configuration under IntelliJ IDEA and VS Code

2.1. Configuration under IntelliJ IDEA

1. Open IntelliJ IDEA.
2. Choose **New Project**
3. Select **Maven**
4. **Java 17** SDK
5. Create a parent project named **digibank-parent**
6. Enable **automatic import** of Maven projects
7. **src/main/java** folder is recognized as the source.
8. Verify that the **src/test/java folder** is recognized as a test folder.
9. Install the plugins
    - Jakarta EE support
    - Maven
    - JUnit
    - Cucumber for Java
    - Database tools if available

2.2. Configuration under VS Code

1. Install the following extensions:
    - Extension Pack for Java
    - Maven for Java
    - Language Support for Java by Red Hat
    - Debugger for Java
    - Test Runner for Java
    - Cucumber (Gherkin) Full Support or equivalent
2. **.vscode/settings.json** file :

```
{
    "java.configuration.runtimes" : [
{
```

```
            "name" : "JavaSE-17",
            "path" : "C:/Program Files/Java/jdk-17"
}
],
        "java.import.maven.enabled" : true ,
        "java.test.config" : {
            "workingDirectory" : "${ workspaceFolder }"
}
}
```

3. On Linux or macOS, replace the JDK path with the actual local path.

```
user@ca:4f:12:a0:b3:c1 ~ % export JAVA_HOME= $(
/usr/libexec/java_home -v 17 )
export PATH = $JAVA_HOME / bin : $PATH
user@ca:4f:12:a0:b3:c1 ~ %
```

2.3. Expected result

The chosen IDE should automatically recognize the Maven project, the JDK, JUnit tests, and Gherkin Cucumber files.

3. Creation of the PostgreSQL database

3.1. Creating the database and the user

In PostgreSQL, execute the following commands:

```
CREATE DATABASE digibank_db ;

CREATE USER digibank_user WITH PASSWORD 'digibank_pwd' ;

GRANT ALL PRIVILEGES ON DATABASE digibank_db TO digibank_user;
```

3.2. Connection Verification

```
psql -U digibank_user -d digibank_db -h localhost
```

3.3. Expected Result

```
psql (18.4 (Postgres.app))

Kind "help" for help.


postgres = # CREATE DATABASE digibank_db;

CREATE DATABASE
```

```
postgres = # CREATE USER digibank_user WITH PASSWORD
'digibank_pwd' ;

CREATE ROLE

postgres = # GRANT ALL PRIVILEGES ON DATABASE digibank_db TO
digibank_user;

GRANT

postgres = #
```

The connection to the **digibank_db database** must be accepted without error.

4. Creation of the multi-module Maven project

4.1. Project Tree Structure

Create the following structure:

```
digibank - parent /

├── pom.xml

├── digibank - shared /

| ├── pom . xml

| └── src /

| ├── main / java / com / digibank / shared /

|└──test / java / com / digibank / shared /

├── digibank - customer /

| ├── pom . xml

| └── src /

| ├── main / java / com / digibank / customer /
```

```
| ├── main / resources / META - INF /
| └── test / java / com / digibank / customer /
├── digibank - account /
| ├── pom.xml
| └── src /
| ├── main / java / com / digibank / account /
| ├── main / resources / META - INF /
| └── test / java / com / digibank / account /
├── digibank - transaction /
| ├── pom.xml
| └── src /
| ├── main / java / com / digibank / transaction /
| ├── main / resources / META - INF /
| └── test / java / com / digibank / transaction /
├── digibank - compliance /
| ├── pom . xml
| └── src /
| ├── main / java / com / digibank / compliance /
| ├── main / resources / META - INF /
| └── test / java / com / digibank / compliance /
├── digibank - index /
| ├── pom.xml
| └── src /
| ├── main / java / com / digibank / index /
| └── test / java / com / digibank / index /
└── digibank - app /
```

```
├── pom.xml
└── src /
└── hand /
├── application / META - INF /
└── webapp /
└── WEB - INF /
```

This structure directly reflects the modular orientation of the Digi Bank system, with a shared module and separate business modules, then a final assembly module for deployment.

4.2. pom.xml parent

digibank-parent/pom.xml file :

```
<project xmlns ="http://maven.apache.org/POM/4.0.0"
        xmlns: xsi ="http://www.w3.org/2001/XMLSchema-instance"
        xsi :schemaLocation ="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd" >
<modelVersion> 4.0.0 </modelVersion>

<groupId> com.digibank </groupId>
<artifactId> digibank-parent </artifactId>
<version> 1.0.0 </version>
<packaging> pom </packaging>

<modules>
```

```
<module> digibank-shared </module>

<module> digibank-customer </module>

<module> digibank-account </module>

<module> digibank-transaction </module>

<module> digibank-compliance </module>

<module> digibank-index </module>

<module> digibank-app </module>

</modules>


<properties>

<maven.compiler.source> 17 </maven.compiler.source>

<maven.compiler.target> 17 </maven.compiler.target>

<project.build.sourceEncoding>                                    UTF-8
</project.build.sourceEncoding>

<jakartaee.version> 10.0.0 </jakartaee.version>

<junit.version> 5.10.2 </junit.version>

<cucumber.version> 7.15.0 </cucumber.version>

<junit.platform.version> 1.10.2 </junit.platform.version>

</properties>


<dependencyManagement>

<dependencies>

<dependency>

<groupId> jakarta.platform </groupId>
```

```
<artifactId> jakarta.jakartaee-api </artifactId>
<version> ${jakartaee.version} </version>
<scope> provided </scope>
</dependency>
</dependencies>
</dependencyManagement>
</project>
```

5. Module creation

5.1. Shared module digibank-shared

1. Create **digibank-shared/pom.xml** :

```
<project xmlns ="http://maven.apache.org/POM/4.0.0"
        xmlns: xsi ="http://www.w3.org/2001/XMLSchema-instance"
        xsi :schemaLocation ="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd
<parent>
<groupId> com.digibank </groupId>
<artifactId> digibank-parent </artifactId>
<version> 1.0.0 </version>
```

```
</parent>


<modelVersion> 4.0.0 </modelVersion>

<artifactId> digibank-shared </artifactId>

<packaging> jar </packaging>


<dependencies>

<dependency>

<groupId> jakarta.platform </groupId>

<artifactId> jakarta.jakartaee-api </artifactId>

<scope> provided </scope>

</dependency>

</dependencies>

</project>
```

2. Create the common utility class **BaseEntity.java:**

```
package com.digibank.shared.model;


import jakarta.persistence. GeneratedValue ;

import jakarta.persistence.GenerationType;

import jakarta.persistence. ID ;

import jakarta.persistence. MappedSuperclass ;
```

```
@MappedSuperclass
public abstract class BaseEntity {


   @Id
@GeneratedValue ( strategy = GenerationType.IDENTITY )
   private Long id ;


   public Long getId () {
       return id ;
}
}
```

3. Create the common utility class **ApiResponse.java:**

```
package com.digibank.shared.dto;


public class ApiResponse {
   private String message ;
   private boolean success ;


   public ApiResponse () {
}
```

```
    public ApiResponse (String message, boolean success) {
        this.message = message ;
        this.success = success ;
}

    public String getMessage () {
        return message ;
}

    public boolean isSuccess () {
        return success ;
}

    public void setMessage (String message) {
        this.message = message ;
}

    public void setSuccess ( boolean success) {
        this.success = success ;
}
}
```

5.2. digibank-customer customer module

1. Create **digibank-customer/pom.xml:**

```
<project xmlns ="http://maven.apache.org/POM/4.0.0"
        xmlns: xsi ="http://www.w3.org/2001/XMLSchema-instance"
        xsi :schemaLocation ="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd" >
<parent>
<groupId> com.digibank </groupId>
<artifactId> digibank-parent </artifactId>
<version> 1.0.0 </version>
</parent>

<modelVersion> 4.0.0 </modelVersion>
<artifactId> digibank-customer </artifactId>
<packaging> jar </packaging>

<dependencies>
<dependency>
<groupId> com.digibank </groupId>
<artifactId> digibank-shared </artifactId>
<version> 1.0.0 </version>
</dependency>
<dependency>
<groupId> jakarta.platform </groupId>
<artifactId> jakarta.jakartaee-api </artifactId>
```

```
<scope> provided </scope>
</dependency>
</dependencies>
</project>
```

2. Create the **Customer entity**

```
package com.digibank.customer.model;

import com.digibank.shared.model.BaseEntity;
import jakarta.persistence. Column ;
import jakarta.persistence. Entity ;
import jakarta.persistence. Table ;

@Entity
@Table (name = "customers" )
public class Customer extends BaseEntity {

   @Column (nullable = false )
   private String firstName ;

   @Column (nullable = false )
   private String lastName ;
```

```
    @Column (nullable = false , unique = true )
    private String email ;

    public Customer () {
}

     public Customer (String firstName, String lastName, String
email) {
        this.firstName = firstName ;
        this.lastName = lastName ;
        this.email = email ;
}

    public String getFirstName () {
        return firstName ;
}

    public String getLastName () {
        return lastName ;
}

    public String getEmail () {
        return email ;
```

```
}

   public void setFirstName (String firstName) {
        this.firstName = firstName ;
}

   public void setLastName (String lastName) {
        this.lastName = lastName ;
}

   public void setEmail (String email) {
        this.email = email ;
}
}
```

3. Create the **CustomerRepository repository**

```
package com.digibank.customer.repository;

import com.digibank.customer.model.Customer;
import jakarta.ejb. Stateless ;
import jakarta.persistence.EntityManager;
import jakarta.persistence. PersistenceContext ;
```

```
import java.util.List;


@Stateless
public class CustomerRepository {


   @PersistenceContext (unitName = "digibankPU" )
   private EntityManager em ;


   public void save (Customer customer) {
       em .persist(customer);
}


   public Customer findById (Long id) {
       return em .find(Customer. class , id);
}


   public List<Customer> findAll () {
         return em .createQuery( " SELECT c FROM Customer c " ,
Customer. class ).getResultList();
}
}
```

4. Create the **CustomerService service**

```
package com.digibank.customer.service;

import com.digibank.customer.model.Customer;
import com.digibank.customer.repository.CustomerRepository;
import jakarta.ejb. Stateless ;
import jakarta.inject. Inject ;

import java.util.List;

@Stateless
public class CustomerService {

   @Inject
   private CustomerRepository repository ;

   public void createCustomer (Customer customer) {
       repository .save(customer);
}

   public Customer getCustomer (Long id) {
       return repository .findById(id);
}
```

```
   public List<Customer> getAllCustomers () {
       return repository.findAll ();
}
}
```

5. Create the REST resource **CustomerResource**

```
package com.digibank.customer.api;

import com.digibank.customer.model.Customer;
import com.digibank.customer.service.CustomerService;
import jakarta.inject. Inject ;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;

@Path ( "/customers" )
@Consumes ( MediaType.APPLICATION_JSON )
@Produces ( MediaType.APPLICATION_JSON )
public class CustomerResource {

   @Inject
```

```
   private CustomerService service ;


   @POST
   public Response create (Customer customer) {
       service .createCustomer(customer);
            return  Response.  status  (Response.Status.CREATED  )
.entity (customer).build();
}


   @GET
@Path ( "/{id}" )
   public Response findById ( @PathParam ( "id" ) Long id) {
Customer customer = service .getCustomer(id);
       if (customer == null ) {
             return Response. status (Response.Status. NOT_FOUND
).build();
}
       return Response. ok (customer).build();
}


   @GET
   public Response findAll () {
              return  Response.  ok  (  service.getAllCustomers
()).build();
}
```

```
}
```

5.3. digibank-account account module

1. Create **digibank-account/pom.xml** using the same template as **customer** , with a dependency on **digibank-shared.**

2. Create the **Account entity**

```
package com.digibank.account.model;

import com.digibank.shared.model.BaseEntity;
import jakarta.persistence. Column ;
import jakarta.persistence. Entity ;
import jakarta.persistence. Table ;

@Entity
@Table (name = "accounts" )
public class Account extends BaseEntity {

   @Column (nullable = false , unique = true )
   private String accountNumber ;

   @Column (nullable = false )
```

```
    private Double balance ;


    public Account () {
}


    public Account (String accountNumber, Double balance) {
        this . accountNumber = accountNumber;
        this.balance = balance ;
}


    public String getAccountNumber () {
        return accountNumber ;
}


    public Double getBalance () {
        return balance ;
}


    public void setAccountNumber (String accountNumber) {
        this . accountNumber = accountNumber;
}


    public void setBalance (Double balance) {
        this.balance = balance ;
```

```
}

}
```

3. Create the **AccountService service (simplified version)**

```
package com.digibank.account.service;


import com.digibank.account.model.Account;

import jakarta.ejb. Stateless ;


import java.util.ArrayList;

import java.util.List;


@Stateless

public class AccountService {


       private   static   final   List<Account>   accounts   =   new
ArrayList<>();


   public void createAccount (Account account) {

       accounts.add (account);

}


   public List<Account> getAccounts () {
```

```
        return accounts ;
}
}
```

4. Create the REST resource **AccountResource:**

```
package com.digibank.account.api;

import com.digibank.account.model.Account;
import com.digibank.account.service.AccountService;
import jakarta.inject. Inject ;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;

@Path ( "/accounts" )
@Consumes ( MediaType.APPLICATION_JSON )
@Produces ( MediaType.APPLICATION_JSON )
public class AccountResource {

   @Inject
   private AccountService service ;

   @POST
   public Response create (Account account) {
```

```
        service .createAccount(account);

            return  Response.  status  (Response.Status.CREATED  )
.entity (account).build();

}


   @GET

   public Response findAll () {

        return Response. ok ( service.getAccounts ()).build();

}

}
```

5.4. digibank-transaction module

1. Create **digibank-transaction/pom.xml** using the same template.

2. Create the **Transaction entity**

```
package com.digibank.transaction.model;


import com.digibank.shared.model.BaseEntity;

import jakarta.persistence. Column ;

import jakarta.persistence. Entity ;

import jakarta.persistence. Table ;
```

```java
@Entity
@Table (name = "transactions" )
public class Transaction extends BaseEntity {

   @Column (nullable = false )
   private String type ;

   @Column (nullable = false )
   private Double amount ;

   public Transaction () {
}

   public Transaction (String type, Double amount) {
       this.type = type ;
       this.amount = amount ;
}

   public String getType () {
       return type ;
}

   public Double getAmount () {
       return amount ;
```

```
}


   public void setType (String type) {

        this.type = type ;

}


   public void setAmount (Double amount) {

        this.amount = amount ;

}

}
```

3. Create the **TransactionService**

```
package com.digibank.transaction.service;


import com.digibank.transaction.model.Transaction;

import jakarta.ejb. Stateless ;


import java.util.ArrayList;

import java.util.List;


@Stateless

public class TransactionService {
```

```
    private static final List<Transaction> transactions = new
ArrayList<>();


   public void addTransaction (Transaction transaction) {
       transactions .add(transaction);
}


   public List<Transaction> getTransactions () {
       return transactions ;
}
}
```

4. Create the REST **TransactionResource**

```
package com.digibank.transaction.api;


import com.digibank.transaction.model.Transaction;
import com.digibank.transaction.service.TransactionService;
import jakarta.inject. Inject ;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;
```

```
@Path ( "/transactions" )
@Consumes ( MediaType.APPLICATION_JSON )
@Produces ( MediaType.APPLICATION_JSON )
public class TransactionResource {

   @Inject
   private TransactionService service ;

   @POST
   public Response create (Transaction transaction) {
       service.addTransaction (transaction);
            return Response. status (Response.Status.CREATED )
.entity (transaction).build();
}

   @GET
   public Response findAll () {
               return Response. ok ( service.getTransactions
()).build();
}
}
```

5.5. Digibank Compliance Module

1. This module can intentionally remain lighter in this workshop, in order to maintain a scope that can be carried out in practical work while respecting the planned Digi Bank structure.

2. Create the **ComplianceService**

```
package com.digibank.compliance.service;

import jakarta.ejb. Stateless ;

@Stateless
public class ComplianceService {

   public boolean validateTransactionAmount (Double amount) {
       return amount != null && amount <= 10000 ;
}
}
```

3. Create the REST resource **ComplianceResource**

```
package com.digibank.compliance.api;
```

```
import com.digibank.compliance.service.ComplianceService;
import jakarta.inject. Inject ;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;


@Path ( "/compliance" )
@Produces ( MediaType.APPLICATION_JSON )
public class ComplianceResource {


   @Inject
   private ComplianceService service ;


   @GET
@Path ( "/validate/{amount}" )
    public Response validate ( @PathParam ( "amount" ) Double
amount) {
                                boolean    valid    =    service
.validateTransactionAmount(amount);
         return Response. ok ( "{ \" valid \" :" + valid + "}"
).build();
}
}
```

5.6. digibank-index index module

1. Create **digibank-index/pom.xml** :

```
<project xmlns ="http://maven.apache.org/POM/4.0.0"
        xmlns: xsi ="http://www.w3.org/2001/XMLSchema-instance"
        xsi :schemaLocation ="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd" >
<parent>
<groupId> com.digibank </groupId>
<artifactId> digibank-parent </artifactId>
<version> 1.0.0 </version>
</parent>

<modelVersion> 4.0.0 </modelVersion>
<artifactId> digibank-index </artifactId>
<packaging> war </packaging>

<dependencies>
<dependency>
<groupId> jakarta.platform </groupId>
<artifactId> jakarta.jakartaee-api </artifactId>
<scope> provided </scope>
```

```
</dependency>

</dependencies>


<build>

<finalName> digibank-index </finalName>

</build>

</project>
```

2. Create the **IndexServlet servlet**

```
package com.digibank.index.servlet;


import jakarta.servlet.ServletException;

import jakarta.servlet.annotation. WebServlet ;

import jakarta.servlet.http.HttpServlet;

import jakarta.servlet.http.HttpServletRequest;

import jakarta.servlet.http.HttpServletResponse;


import java.io.IOException;


@WebServlet (urlPatterns = { "/" , "/index" })

public class IndexServlet extends HttpServlet {
```

```
   @Override
        protected void doGet (HttpServletRequest request,
HttpServletResponse response)
           throws ServletException, IOException {
       // Redirect to the JSP
                    request.getRequestDispatcher( "/index.jsp"
).forward(request, response);
}
}
```

3. Create the jsp page **main/webapp/index.jsp**

```
<%@ page contentType= "text/html;charset=UTF-8" language= "java"
%>
<!DOCTYPE html>
<html lang= "fr" >
<head>
<meta charset= "UTF-8" >
<meta name= "viewport" content= "width=device-width,
initial-scale=1.0" >
<title>Digi Bank - Home</title>
<style>
* {
margin: 0 ;
```

```
padding: 0 ;

box-sizing: border-box;

}


body {

font-family: 'Segoe UI' , Tahoma, Geneva, Verdana, sans-serif;

background: linear-gradient( 135 deg, # 667 eea 0 %, # 764 ba2
100 %);

min-height: 100 vh;

display: flex;

justify-content: center;

align-items: center;

padding: 20px ;

}


.container {

background: white;

border-radius: 10px ;

box-shadow: 0 10 px 40 px rgba( 0 , 0 , 0 , 0.2 );

max-width: 800px ;

width: 100 %;

padding: 50px ;

text-align: center;

}
```

```
.logo {
font-size: 48px ;
margin-bottom: 20px ;
color: # 667 eea;
}

h1 {
color: # 333 ;
margin-bottom: 20px ;
font-size: 36px ;
}

.version {
color: # 888 ;
font-size: 14px ;
margin-bottom: 30px ;
}

.description {
color: # 555 ;
font-size: 16px ;
line-height: 1.8 ;
margin : 30px 0 ;
```

```
text-align: justify;

}


.features {

background: #f5f5f5;

padding: 30px ;

border-radius: 8px ;

margin : 30px 0 ;

text-align: left;

}


.features h3 {

color: # 667 eea;

margin-bottom: 15px ;

}


.features ul {

list-style: none;

padding: 0 ;

}


.features li {

padding : 8px 0 ;

color: # 555 ;
```

```
border-bottom: 1 px solid #ddd;

}


.features li:last-child {

border-bottom: none;

}


.features li:before {

content: "✓ " ;

color: # 667 eea;

font-weight: bold;

margin-right: 10px ;

}


.links {

margin: 40px 0 0 0 ;

display: flex;

gap: 15px ;

justify-content: center;

flex-wrap: wrap;

}


.btn {

display: inline-block;
```

```
padding : 12px 30px ;

border-radius: 5px ;

text-decoration: none;

font-weight: 600 ;

transition: all 0 . 3 s ease;

cursor: pointer;

border: none;

font-size: 14px ;

}


.btn-primary {

background: linear-gradient( 135 deg, # 667 eea 0 %, # 764 ba2
100 %);

color: white;

}


.btn-primary:hover {

transform: translateY(- 2 px);

box-shadow: 0 5 px 20 px rgba( 102 , 126 , 234 , 0.4 );

}


.btn-secondary {

background: white;

color: # 667 eea;
```

```
border: 2px solid # 667 eea;

}


.btn-secondary:hover {

background: #f5f5f5;

transform: translateY(- 2 px);

}


.api-info {

background: #e8f4f8;

padding: 20px ;

border-left: 4 px solid # 667 eea;

margin : 30px 0 ;

text-align: left;

border-radius: 5px ;

}


.api-info h4 {

color: # 667 eea;

margin-bottom: 10px ;

}


.api-info p {

color: # 555 ;
```

```
font-size: 14px ;

font-family: 'Courier New' , minivan;

margin : 5px 0 ;

}


footer {

margin-top: 40px ;

padding-top: 30px ;

border-top: 1px solid #eee;

color: # 888 ;

font-size: 12px ;

}

</style>

</head>

<body>

<div class= "container" >

<div class= "logo" >🏦</div>

<h1>Digi Bank</h1>

<div class= "version" >Version 1.0 - Workshop 2 </div>


<div class= "description" >

<p>

<strong>Digi  Bank</strong>  is  a  digital  financial  management
platform based on an architecture
```

```
Modular monolithic. It offers an integrated solution for
managing customers, accounts,

transactions and compliance checks.

</p>

</div>


<div class= "features" >

<h3>Available Modules</h3>

<ul>

<li>Customer Management (Customer Module)</li>

<li>Account Management (Account Module)</li>

<li>Transaction Management (Transaction Module)</li>

<li>Compliance Controls (Compliance Module)</li>

<li>Shared Services (Shared Module)</li>

</ul>

</div>


<div class= "api-info" >

<h4>📡 Access to REST APIs</h4>

<p><strong>Base URL:</strong> http://localhost: 8080
/digibank-app/api/</p>

<p><strong>Available endpoints:</strong></p>

<p>• GET /api/index - General Information</p>

<p>• POST /api/customers - Create a customer</p>

<p>• GET /api/customers - List customers</p>
```

```
<p>• POST /api/accounts - Create an account</p>

<p>• GET /api/accounts - List accounts</p>

<p>• POST /api/transactions - Create a transaction</p>

<p>• GET /api/transactions - List transactions</p>

<p>• GET /api/compliance/validate/{amount} - Validate an amount</p>

</div>


<div class= "links" >

<a href= "http://localhost:8080/digibank-app/api/" class= "btn btn-primary" > REST API</a>

<a href= "<%= request.getContextPath() %> /docs" class= "btn btn-secondary" >Documentation (Soon)</a>

</div>


<footer>

<p>Professional Bachelor's Degree in Cloud Computing - University of the Mountains</p>

<p>Supervisor: Eng. Willy Damtchou | June 2026 </p>

</footer>

</div>

</body>

</html>
```

6. Common Jakarta EE setup

6.1. JAX-RS Activation Class

1. Create in **digibank-app:**

```
package com.digibank.app;

import jakarta.ws.rs. ApplicationPath ;
import jakarta.ws.rs.core.Application;

@ApplicationPath ( "/api" )
public class DigiBankApplication extends Application {
}
```

6.2. persistence.xml

Create **META-INF/persistence.xml** in the assembly module, i.e. **digibank-app/src/main/resources/META-INF/** depending on the chosen organization:

```
<persistence xmlns ="https://jakarta.ee/xml/ns/persistence"
             version ="3.0" >
<persistence-unit name ="digibankPU" transaction-type ="JTA" >
```

```
<jta-data-source> java:/jdbc/DigiBankDS </jta-data-source>

<properties>

<property                                                    name
="jakarta.persistence.schema-generation.database.action"    value
="update" />

</properties>

</persistence-unit>

</persistence>
```

7. digibank-app assembly module

7.1. pom.xml of the application module

```
<project xmlns ="http://maven.apache.org/POM/4.0.0"

        xmlns: xsi ="http://www.w3.org/2001/XMLSchema-instance"

        xsi :schemaLocation ="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd

<parent>

<groupId> com.digibank </groupId>

<artifactId> digibank-parent </artifactId>

<version> 1.0.0 </version>

</parent>
```

```
<modelVersion> 4.0.0 </modelVersion>
<artifactId> digibank-app </artifactId>
<packaging> war </packaging>

<dependencies>
<dependency>
<groupId> com.digibank </groupId>
<artifactId> digibank-shared </artifactId>
<version> 1.0.0 </version>
</dependency>
<dependency>
<groupId> com.digibank </groupId>
<artifactId> digibank-customer </artifactId>
<version> 1.0.0 </version>
</dependency>
<dependency>
<groupId> com.digibank </groupId>
<artifactId> digibank-account </artifactId>
<version> 1.0.0 </version>
</dependency>
<dependency>
<groupId> com.digibank </groupId>
<artifactId> digibank-transaction </artifactId>
```

```
<version> 1.0.0 </version>

</dependency>

<dependency>

<groupId> com.digibank </groupId>

<artifactId> digibank-compliance </artifactId>

<version> 1.0.0 </version>

</dependency>

<dependency>

<groupId> com.digibank </groupId>

<artifactId> digibank-index </artifactId>

<version> 1.0.0 </version>

<type> war </type>

</dependency>

<dependency>

<groupId> jakarta.platform </groupId>

<artifactId> jakarta.jakartaee-api </artifactId>

<scope> provided </scope>

</dependency>

</dependencies>


<build>

<finalName> digibank-app </finalName>

<plugins>

<plugin>
```

```
<groupId> org.apache.maven.plugins </groupId>
<artifactId> maven-war-plugin </artifactId>
<version> 3.4.0 </version>
<configuration>
<overlays>
<overlay>
<groupId> com.digibank </groupId>
<artifactId> digibank-index </artifactId>
</overlay>
</overlays>
</configuration>
</plugin>
</plugins>
</build>
</project>
```

8. WildFly configuration

8.1. Downloading Wildfly 33.0.2.Final

Recommended version: **33.0.2.Final**

Download link:
https://github.com/wildfly/wildfly/releases/download/33.0.2.Final/wildfly-33.0.2.Final.zip

8.2. Downloading the PostgreSQL driver

Recommended version: **postgresql-42.7.x.jar**

Download link: **https://jdbc.postgresql.org/download/**

8.3. Creating the WildFly module

In WildFly, create the following directory if necessary.

```
WILDFLY_HOME/modules/system/layers/base/org/postgresql/main/
```

Place inside:

- postgresql-42.7.x.jar
- module.xml (Enter the exact name of the postgresql.jar file)

```
<? xml version = "1.0" encoding = "UTF-8" ?>

< module xmlns = "urn:jboss:module:1.5" name = "org.postgresql"
>

   < resources >

      < resource-root path = "postgresql-42.7.11.jar" />

   </resources>

   < dependencies >

      < module name = "javax.api" />
```

```
        < module name = "javax.transaction.api" />

    </dependencies>

</module>
```

8.4. DigiBank datasource declaration in WildFly

1. open **standalone.xml**

```
WILDFLY_HOME/standalone/configuration/standalone.xml
```

2. In the **<datasources> section,** add:

```
< datasource jndi-name = "java:/jdbc/DigiBankDS" pool-name =
"DigiBankDS" enabled = "true" use-java-context = "true" >

    < connection-url >
jdbc:postgresql://localhost:5432/digibank_db </ connection-url >

    < driver > postgresql </ driver >

    < security user-name = "digibank_user" password =
"digibank_pwd" />

</datasource>
```

3. **<drivers>** subsection of **<datasources>** , add:

```
< driver name = "postgresql" module = "org.postgresql" >
   < driver-class > org.postgresql.Driver </ driver-class >
</driver>
```

8.5. Starting WildFly and verifying the datasource

a. Booting under Linux or macOS

```
/path/to/wildfly/bin/standalone.sh
```

b. Booting under Linux or Windows

```
/path/to/wildfly/bin/standalone.bat
```

c. Expected result

WildFly must start with the datasource **java:/jdbc/DigiBankDS** active and without connection errors. You should see the following in the WildFly startup logs: **Bound data source [java:/jdbc/DigiBankDS]**

```
03:43:17,824 INFO [ org.wildfly.extension.undertow ] (MSC service thread 1-4)
WFLYUT0006: Undertow HTTPS listener https listening on 127.0.0.1:8443

03:43:17,839 INFO [ org.jboss.as.connector.subsystems.datasources ] (MSC service
thread 1-6) WFLYJCA0001: Bound data source [ java:/jdbc/DigiBankDS ]

03:43:17,839 INFO [ org.jboss.as.connector.subsystems.datasources ] (MSC service
thread 1-7) WFLYJCA0001: Bound data source [ java:jboss/datasources/ExampleDS ]

03:43:17,899 INFO [ org.jboss.as.server.deployment.scanner ] (MSC service thread 1-4)
WFLYDS0013: Started FileSystemDeploymentService for directory
/Users/user/Documents/Cours UDM/Cloud
Architecture/Code/wildfly-33.0.2.Final/standalone/deployments

03:43:17,933 INFO [ org.jboss.ws.common.management ] (MSC service thread 1-6)
JBWS022052: Starting JBossWS 7.1.0.Final (Apache CXF 4.0.5)

03:43:17,967 INFO [ org.jboss.as.server ] (Controller Boot Thread) WFLYSRV0212:
Resuming server
```

9. Scripts and commands to execute

9.1. Complete Compilation

From the **digibank-parent root** :

```
mvn clean install
```

9.2. WAR Generation

```
mvn -pl digibank-app clean package
```

9.3. Deployment on WildFly via copy

A simple method is to copy the generated WAR file into the WildFly deployment folder:

```
cp digibank-app/target/digibank-app.war
/path/to/wildfly/standalone/deployments/
```

9.4. Deployment on WildFly by Maven

1. **Create an admin user for the Wildfly console**

This is done using the **add-user command.**

```
/path/to/wildfly/bin/add-user.sh/
```

Choose :

- Type: **Management User**
- username: **admin**
- password: **admin**

2. **Start WildFly before the Maven deployment**

3. **Configure the wildfly-maven-plugin in the assembly module**

a. In the **digibank-app module** , add the **wildfly-maven- plugin to the <build>** section of the pom.xml:

```
<build>
<finalName> digibank-app </finalName>
<plugins>
<plugin>
<groupId> org.apache.maven.plugins </groupId>
<artifactId> maven-war-plugin </artifactId>
<version> 3.4.0 </version>
<configuration>
<overlays>
<overlay>
<groupId> com.digibank </groupId>
<artifactId> digibank-index </artifactId>
</overlay>
</overlays>
</configuration>
</plugin>
<plugin>
<groupId> org.wildfly.plugins </groupId>
<artifactId> wildfly-maven-plugin </artifactId>

<configuration>
<skip> false </skip>
<hostname> localhost </hostname>
<port> 9990 </port>
```

```
<username> admin </username>
<password> admin </password>
<filename> digibank-app.war </filename>
</configuration>
</plugin>
</plugins>
</build>
```

b. In the **digibank-parent root directory** , add the **<build> section** to the **pom.xml file** :

```
<build>
<pluginManagement>
<plugins>
<plugin>
<groupId> org.wildfly.plugins </groupId>
<artifactId> wildfly-maven-plugin </artifactId>
<version> 5.0.0.Final </version>
<configuration>
<skip> true </skip>
</configuration>
</plugin>
</plugins>
</pluginManagement>
```

```
</build>
```

By centralizing the configuration of the WildFly plugin and enabling skip=true by default, only the digibank-app module is deployed, thus avoiding the unnecessary deployment of all the project's modules.

4. **Deploy automatically with Maven**

From the project's origin:

```
mvn clean install wildfly:deploy
```

content repository system to manage deployments:

- The deployment metadata is located in: **standalone/configuration/**
- The actual content of the WAR file is stored with a SHA1 hash in: **standalone/data/content/**
- **standalone/deployments/** directory remains empty because it is in **managed mode.**

9.5. Expected result

The application must be deployed and accessible via the local server URL

http://ip:port_wildfly/digibank-app

10. Functional checks

10.1. Endpoint Testing

Test the following entry points:

- **GET /api/index**
- **POST /api/customers**
- **GET /api/customers**
- **POST /api/accounts**
- **GET /api/accounts**
- **POST /api/transactions**
- **GET /api/transactions**
- **GET /api/compliance/validate/5000**
- **GET /api/compliance/validate/25000**

10.2. Example curl commands

An example using **curl:**

```
curl http://localhost:8080/digibank-app/api/index
```

An example of a curl command for creating a client:

```
curl -X POST http://localhost:8080/digibank-app/api/customers \
-H "Content-Type: application/json" \
-d "{ \" firstName \" : \" Ali \" , \" lastName \" : \" Diallo
\" , \" email \" : \" ali.diallo@digibank.com \" }"
```

10.3. Expected Result

The endpoints must respond correctly and reflect the basic business logic implemented in each module.

11. JUnit Unit Tests

11.1. Test Dependencies

Add to the relevant modules:

```
<dependency>
<groupId> org.junit.jupiter </groupId>
<artifactId> junit-jupiter </artifactId>
<version> ${junit.version} </version>
<scope> test </scope>
</dependency>
```

11.2. Compliance Service Test

ComplianceServiceTest.java test class :

```
package com.digibank.compliance.service;
```

```
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api. Test ;

public class ComplianceServiceTest {

        private final ComplianceService service = new
ComplianceService();

   @Test
   void shouldAcceptAmountBelowLimit () {
Assertions. assertTrue ( service.validateTransactionAmount (
5000.0 ));
}

   @Test
   void shouldRejectAmountAboveLimit () {
Assertions. assertFalse ( service.validateTransactionAmount (
20000.0 ));
}
}
```

11.3. Test execution

```
mvn test
```

11.4. Expected Result

The tests must run without errors and confirm the expected behavior of the compliance service.

12. Database testing with Cucumber

12.1. Maven Dependencies

Add to the relevant test module:

```
<dependency>
<groupId> org.junit.platform </groupId>
<artifactId> junit-platform-suite-api </artifactId>
<version> ${junit.platform.version} </version>
<scope> test </scope>
</dependency>
<dependency>
<groupId> io.cucumber </groupId>
<artifactId> cucumber-java </artifactId>
<version> ${cucumber.version} </version>
<scope> test </scope>
</dependency>
```

```
<dependency>
<groupId> io.cucumber </groupId>
<artifactId> cucumber-junit-platform-engine </artifactId>
<version> ${cucumber.version} </version>
<scope> test </scope>
</dependency>
```

12.2. Feature File

Create the file **src/test/resources/features/compliance.feature** :

```
Feature : Transaction amount validation

 Scenario : Amount as agreed
   Given a transaction amount of 5000
   When the compliance check is performed
   Then the transaction is accepted

 Scenario : Incorrect amount
   Given a transaction amount of 20,000
   When the compliance check is performed
   Then the transaction is rejected
```

12.3. Definition of the Steps

Create **ComplianceSteps.java** :

```
package com.digibank.compliance.steps;

import com.digibank.compliance.service.ComplianceService;
import io.cucumber.java.en. Given ;
import io.cucumber.java.en. Then ;
import io.cucumber.java.en. When ;
import org.junit.jupiter.api.Assertions;

public class ComplianceSteps {

        private final ComplianceService service = new
ComplianceService();
   private Double amount ;
   private boolean result ;

   @Given ( "a transaction amount of {int}" )
   public void aTransactionAmountOf (Integer amount) {
       this . amount = amount.doubleValue();
}

   @When ( "the compliance check is executed" )
```

```
    public void ComplianceCheckIsExecute () {
        result = service.validateTransactionAmount ( amount );
}

    @Then ( "the transaction is accepted" )
    public void theTransactionIsAccepted () {
Assertions. assertTrue ( result );
}

    @Then ( "the transaction is rejected" )
    public void theTransactionIsRejected () {
Assertions. assertFalse ( result );
}
}
```

12.4. Runner Cucumber

Create **the RunCucumberTest.java runner** :

```
package com.digibank.compliance;

import org.junit.platform.suite.api.IncludeEngines;
import org.junit.platform.suite.api.SelectClasspathResource;
import org.junit.platform.suite.api.Suite;
```

```
@Following
@IncludeEngines ( "cucumber" )
@SelectClasspathResource ( "features" )
public class RunCucumberTest {
}
```

12.5. Expected result

Cucumber scenarios should allow verification of business behavior in a form close to functional requirements expressed in natural language.

13. Work assigned to students

Students must complete all of the following tasks:

1. Install the complete development environment.

2. Configure IntelliJ IDEA or VS Code.

3. Create the PostgreSQL database and the application user.

4. Build the multi-module Maven structure.

5. Create all the **pom.xml files.**

6. Develop the **shared, customer, account, transaction, compliance and index modules.**

7. Configure JPA and the datasource.

8. Assemble the application in the **digibank-app module.**

9. Compile, package and deploy the application.

10. Test the REST endpoints.

11. Implement JUnit tests.

12. Implement the Cucumber scenarios.

13. Correct any errors until a stable executable version is obtained.

14. Expected deliverables

Students must submit:

1. **digibank-parent** project
2. all Maven modules created
3. Configuration files
4. SQL scripts for database creation
5. Java classes
6. REST resources
7. JUnit tests
8. The Cucumber scenarios
9. A short note explaining the order of development followed and the main difficulties encountered.

15. Evaluation criteria

The evaluation will be based on the following criteria:

1. Maven tree structure consistency
2. Accuracy of technical configurations
3. Good modular cutting
4. Quality of the Java code produced
5. Validity of REST resources

6. How persistence works
7. Successful execution under WildFly
8. Quality and relevance of JUnit tests
9. Quality and relevance of Cucumber scenarios
10. Respect for the logical order and method of execution.

16. Proposed pricing schedule

1. Environment installation and configuration: 10%
2. Multi-module Maven structure: 15%
3. Business module development: 30%
4. PostgreSQL and WildFly configuration: 15%
5. REST endpoints and application execution: 10%
6. JUnit tests: 10%
7. Cucumber scenarios: 10%.

17. Expected results

At the end of the workshop, the student should have a first executable version of Digi Bank, built around a coherent modular monolith, including the essential functional areas, a database connection, basic business services, operational REST endpoints and a first level of technical and functional testing.

This step directly prepares the rest of the work, which may focus on enriching the business domain, security, technical documentation, improving transactional robustness and increasing software quality.